

Translated

From: e-maxx.ru

Lowest Common Ancestor - Tarjan's off-line algorithm

We have a tree G with n nodes and we have m queries of the form (u, v) . For each query (u, v) we want to find the lowest common ancestor of the vertices u and v , i.e. the node that is an ancestor of both u and v and has the greatest depth in the tree. The node v is also an ancestor of v , so the LCA can also be one of the two nodes.

In this article we will solve the problem off-line, i.e. we assume that all queries are known in advance, and we therefore answer the queries in any order we like. The following algorithm allows to answer all m queries in $O(n + m)$ total time, i.e. for sufficiently large m in $O(1)$ for each query.

Algorithm

The algorithm is named after Robert Tarjan, who discovered it in 1979 and also made many other contributions to the [Disjoint Set Union](#) data structure, which will be heavily used in this algorithm.

The algorithm answers all queries with one [DFS](#) traversal of the tree. Namely a query (u, v) is answered at node u , if node v has already been visited previously, or vice versa.

So let's assume we are currently at node v , we have already made recursive DFS calls, and also already visited the second node u from the query (u, v) . Let's learn how to find the LCA of these two nodes.

Note that $\text{LCA}(u, v)$ is either the node v or one of its ancestors. So we need to find the lowest node among the ancestors of v (including v), for which the node u is a descendant. Also note that for a fixed v the visited nodes of the tree split into a set of disjoint sets. Each ancestor p of node v has his own set containing this node and all subtrees with roots in those of its children who are not part of the path from v to the root of the tree. The set which contains the node u determines the $\text{LCA}(u, v)$: the LCA is the representative of the set, namely the node on lies on the path between v and the root of the tree.